```
      chsrlinf(True)
      sleep(tdly)
      chsrlinb(True)
      i += 1

    i = randint(1,6)
    sleep(2 * tdly)

    if i == 1:
      rledon()
    elif i == 2:
      yledon()
    elif i == 3:
      gledon()
    elif i == 4:
      bledon()
    elif i == 5:
      wledon()
    elif i == 6:
      aledon()

    sleep(50 * tdly)

    if onet:
      aledoff()
      break
  except:
    aledoff()
    if -1 == pfrm.
find('ESP32C3'):
      soft_reset()

randice()

#--- End of chaseleds.py ---
```

You can assemble the entire circuit on a mini breadboard. The code indicates that we must connect the positive pins of the red, yellow, green, blue, and white LEDs to the Super Mini GPIO pins 0, 1, 2, 3, 4, respectively. The negative pins of the LEDs are connected to the GND pin of the board (with or without a 220 ohms current limiting resistor). I have soldered the bigger 10mm LEDs on a PCB strip to make it a permanent arrangement ready to serve as a traffic light module as well when using only the first three LEDs. The connected circuit of our electronic dice is shown in Figure 4.

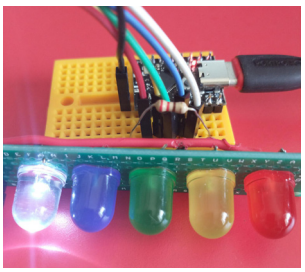


Figure 4: Super Mini-based LEDs dice

Now let's use some new sensors, not covered in the earlier article in September, for home automation. The code given below shows how to use a mini IR pyroelectric infrared PIR motion human sensor to create something like a motion-enabled IoT switch.

```
#--- Start of pirmtnsnsr.
py ---

from machine import Pin,
soft_reset, Signal
from os import uname
from time import sleep

pfrm = uname()[-1].split()
[-1].replace('-','')
ahgh = True

if -1 == pfrm.find('ESP32C3'):
  pirs = Pin(4, Pin.IN, Pin.
PULL_DOWN)
else:
  pirs = Pin(4, Pin.IN, Pin.
PULL_UP)

rgbl = {
        'red' :
Signal(Pin(0, Pin.OUT),
invert=not(ahgh)),
         'grn' :
Signal(Pin(1, Pin.OUT),
invert=not(ahgh)),
         'blu' :
Signal(Pin(3, Pin.OUT),
invert=not(ahgh)),
       }

def rledon():
  rgbl['red'].on()
  rgbl['grn'].off()
  rgbl['blu'].off()


def gledon():
  rgbl['red'].off()
  rgbl['grn'].on()
  rgbl['blu'].off()

def bledon():
  rgbl['red'].off()
  rgbl['grn'].off()
  rgbl['blu'].on()

def aledoff():
  rgbl['red'].off()
  rgbl['grn'].off()
  rgbl['blu'].off()

dly = 5
pln = 0.5

def mtnhdlr(pin):
  if pin.value():
    rledon()
    sleep(dly)
    bledon()
    sleep(1)

pirs.irq(trigger=Pin.IRQ_
RISING, handler=mtnhdlr)

aledoff()

while True:
  try:
    if not pirs.value():
      gledon()

    sleep(pln)
  except:
    aledoff()
    if -1 == pfrm.
find('ESP32C3'):
      soft_reset()

#--- End of pirmtnsnsr.py ---
```

This is a simple circuit to assemble. The VIN/+ve of the sensor is connected to the 3V3 pin of the board, while GND/-ve of the sensor is connected to the GND of the board and the GPIO 3 pin.

Please note, we have intentionally used MicroPython IRQ functionality to demonstrate its usage. MicroPython Interrupt Requests (IRQ) allow your microcontroller to instantly pause its main code and execute a specific function when a hardware event (like a button press or sensor trigger) occurs. This removes the need for wasteful polling in a constant loop. The red, green, blue LED +ve pins are connected to GPIO 0, 1, 3, respectively of the Super Mini. The assembled breadboard circuit is shown in Figure 5. I used an RGB chip LED instead of separate red, green, blue LEDs to keep it small and smart for future needs. Just make a motion around the sensor after running the code and you'll see the chip LED turning to Red for some time, briefly turning to Blue for a moment, to turn Green again until the next motion is detected. You could also connect relay(s) to the LED(s) pin(s) introduced in the last article in September, to turn on and